

- 在父进程中继续执行。控制返回用户态下父进程进行 fork 调用处。
- 处理器控制权交给子进程。子进程开始执行代码，执行点与父进程相同，也就是说在 fork 调用的返回处。
- 控制转交给另一个进程。父进程和子进程都置于就绪状态。

很难想象这种创建进程的方法中父进程和子进程都执行相同的代码。其区别在于：当从 fork 中返回时，测试返回参数，如果值为零，则它是子进程，可以转移到相应的用户程序中继续执行；如果值不为零，则它是父进程，继续执行主程序。

3.8 小结

现代操作系统中最基本的构件是进程，操作系统的基本功能是创建、管理和终止进程。当进程处于活跃状态时，操作系统必须设法使每个进程都分配处理器执行时间，并协调它们的活动、管理有冲突的请求、给进程分配系统资源。

为执行进程管理功能，操作系统维护着对每个进程的描述，或者称为进程映像，它包括执行进程的地址空间和一个进程控制块。进程控制块含有操作系统管理进程所需要的所有信息，包括它的当前状态、分配给它的资源、优先级和其他相关数据。

在整个生命周期中，进程总是在一些状态之间转换。最重要的状态有就绪态、运行态和阻塞态。一个就绪态进程是指当前没有执行但已做好了执行准备的进程，只要操作系统调度到它就立即可以执行；运行态进程是指当前正在被处理器执行的进程，在多处理器系统中，会有多个进程处于这种状态；阻塞态进程正在等待某一事件的完成，如一次 I/O 操作。

一个正在运行的进程可被一个在进程外发生且被处理器识别的中断事件打断，或者被操作系统调用的系统调用打断。不论哪种情况，处理器都执行一次模式切换，把控制转交给操作系统例程。操作系统在完成必需的操作后，可以恢复被中断的进程或者切换到别的进程。

3.9 推荐读物

关于 UNIX 进程管理的详细描述请参阅 [GOOD94] 和 [GRAY97]。[NEHM75] 中有关于进程状态的讨论以及分派所需要的操作系统原语。

GOOD94 Goodheart, B., and Cox, J. *The Magic Garden Explained: The Internals of UNIX System V Release 4*. Englewood Cliffs, NJ: Prentice Hall, 1994.

GRAY97 Gray, J. *Interprocess Communications in UNIX: The Nooks and Crannies*. Upper Saddle River, NJ: Prentice Hall, 1997.

NEHM75 Nehmer, J. "Dispatcher Primitives for the Construction of Operation System Kernels" *Acta Informatica*, vol. 5, 1975.

3.10 关键术语、复习题和习题

关键术语

阻塞态	父进程	进程切换	交换
子进程	抢占	程序状态字	系统态
退出态	特权态	就绪态	任务
中断	进程	轮转	跟踪
内核态	进程控制块	运行态	陷阱
模式切换	进程映像	挂起态	用户态
新建态			

复习题

- 3.1 什么是指令跟踪?
- 3.2 通常有哪些事件会导致创建一个进程?
- 3.3 对于图 3.6 中的进程模型, 请简单定义每个状态。
- 3.4 抢占一个进程是什么意思?
- 3.5 什么是交换, 其目的是什么?
- 3.6 为什么图 3.9b 中有两个阻塞态?
- 3.7 列出挂起进程的 4 个特点。
- 3.8 对于哪类实体, 操作系统为了管理它而维护其信息表?
- 3.9 列出进程控制块中的三类信息。
- 3.10 为什么需要两种模式 (用户态和内核态)?
- 3.11 操作系统创建一个新进程所执行的步骤是什么?
- 3.12 中断和陷阱有什么区别?
- 3.13 举出中断的三个例子。
- 3.14 模式切换和进程切换有什么区别?

习题 注: 图3.9b指7状态模型。

- 3.1 给出操作系统进行进程管理时的 5 种主要活动, 并简单描述为什么需要它们。
- 3.2 假设一个计算机有 A 个输入输出设备和 B 个处理器, 在任何时候内存最多容纳 C 个进程。假设 $A < B < C$, 试问:
 - a) 任一时刻, 处于就绪态、运行态、阻塞态、就绪挂起态、阻塞挂起态的最大进程数目各是多少?
 - b) 处在就绪态、运行态、阻塞态、就绪挂起态、阻塞挂起态的最小进程数目各是多少?
- 3.3 图 3.9b 包含 7 个状态。假设我们让系统有两个就绪状态: 一般就绪状态 READY 和 SYSTEM READY 状态。系统当中处于 SYSTEM READY 状态的进程拥有最高优先级, 并且按轮转方式执行; CPU 调度算法让处于这种状态的进程拥有特权并且它们从不被交换出内存。请画出对应的状态转换图, 标识出每一个状态转换。
- 3.4 假设我们是一个操作系统的开发人员, 该操作系统采用五状态进程模型 (如图 3.5 所示)。在该操作系统中, 所有进程按照轮转方式执行。再假设我们正在为运行 I/O 密集型任务的计算机开发新的操作系统。为了保证这些任务在需要时 (即当一个突发 I/O 完成时) 能快速获得处理器, 我们在进程模型中考虑两个 READY 状态: READY-CPU 状态 (为 CPU 密集型进程) 和 READY-I/O 状态 (为 I/O 密集型进程)。系统中处于 READY-I/O 状态的进程拥有最高权限访问 CPU, 并且以 FCFS 方式执行。处于 READY-CPU 状态的进程以轮转方式执行。当进程进入系统时, 它们被分类为 CPU 密集型或者 I/O 密集型。
 - a) 讨论用如上所描述的两种就绪状态实现操作系统的优点和缺点。
 - b) 讨论如何通过五状态进程模型并修改进程调度算法来获得六状态进程模型的好处。
- 3.5 对于图 3.9b 中给出的 7 状态进程模型, 请仿照图 3.8b 画出它的排队图。
- 3.6 考虑图 3.9b 中的状态转换图。假设操作系统正在分派进程, 有进程处于就绪态和就绪/挂起态, 并且至少有一个处于就绪/挂起态的进程比处于就绪态的所有进程的优先级都高。有两种极端的策略: 1) 总是分派一个处于就绪状态的进程, 以减少交换; 2) 总是把机会给具有最高优先级的进程, 即使会导致在不需要交换时进行交换。请给出一种能均衡考虑优先级和性能的中间策略。
- 3.7 表 3.9 展示了 UNIX SVR4 操作系统的 9 个进程状态。
 - a) 列出表 3.9 中的 9 个状态并且指出这 9 个状态与图 3.9b 中 7 个状态之间的关系。
 - b) 给出 UNIX SVR4 操作系统中存在两个运行状态的理由。
- 3.8 VAX/VMS 操作系统采用了四种处理器访问模式, 以促进系统资源在进程间的保护和共享。访问模式确定:
 - 指令执行特权: 处理器将执行什么指令。
 - 内存访问特权: 当前指令可能访问虚拟内存中的哪个单元。

4 种模式如下：

- 内核模式：执行 VMS 操作系统的内核，包括内存管理、中断处理和 I/O 操作。
- 执行模式：执行许多操作系统服务调用，包括文件（磁盘和磁带）和记录管理例程。
- 管理模式：执行其他操作系统服务，如响应用户命令。
- 用户模式：执行用户程序和诸如编译器、编辑器、链接程序、调试器之类的实用程序。

在较少特权模式执行的进程通常需要调用在较多特权模式下执行的过程，例如，一个用户程序需要一个操作系统服务。这个调用通过使用一个改变模式（简称 CHM）指令来实现，该指令将引发一个中断，把控制转交给处于新的访问模式下的例程，并通过执行 REI（Return from Exception or Interrupt，从异常或中断中返回）指令返回。

- a) 很多操作系统有两种模式，内核态和用户态，那么提供 4 种模式有什么优点和缺点？
b) 你可以举出一种有 4 种以上模式的情况吗？

表 3.13 VAX/VMS 的进程状态

进程状态	说 明
当前正在执行	运行进程
可计算（驻留）	就绪并驻留在内存中
可计算（换出）	就绪但换出内存
页面失效等待	进程引用了不在内存中的页，必须等待读入该页
页冲突等待	程序引用了另一个正处于页面失效等待的进程所等待的共享页，或者引用了进程正在读入或写出的私有页
一般事件等待	等待共享事件标志（事件标志是单位进程间的信号机制）
自由页等待	等待内存中的一个自由页被加入到用于该进程的页集合中（进程的工作页面组）
休眠等待（驻留）	进程把自己置于等待状态
休眠等待（换出）	休眠进程被换出内存
本地事件等待（驻留）	进程在内存中，并正在等待局部事件标志（通常是 I/O 完成）
本地事件等待（换出）	处于本地事件等待状态的进程被换出内存
挂起等待（驻留）	进程被另一个进程置于等待状态
挂起等待（换出）	挂起进程被换出内存
资源等待	进程正在等待各种系统资源

3.9 在前面的问题中讨论的 VMS 方案常常称做环状保护结构，如图 3.18 所示。3.3 节所描述的简单的内核/用户方案是一种两环结构，[SILB04] 指出了这种方法的问题：

环状（层次）结构的主要缺点是它不允许我们实施须知原理，特别地，如果一个对象必须在域 D_i 中可访问，但在域 D_i 中不可访问，则必须有 $j < i$ 。这意味着在 D_i 中可访问的每个段在 D_j 中都可以访问。

- a) 请清楚地解释上面引文中提出的问题。
b) 请给出环状结构操作系统解决这个问题的一种方法。

3.10 图 3.8b 表明一个进程每次只能在一个事件队列中。

- a) 是否能够允许进程同时等待一个或多个事件？
请举例说明。
b) 在这种情况下，如何修改图中的排队结构以支持这个新特点？

3.11 所有现代操作系统都有系统中断。

- a) 请解释中断如何支持多道程序设计。
b) 请解释中断如何支持错误处理。
c) 对于单线程进程，举例说明一个能够引起中断并且导致进程切换的情景。另外请举例说明能够引

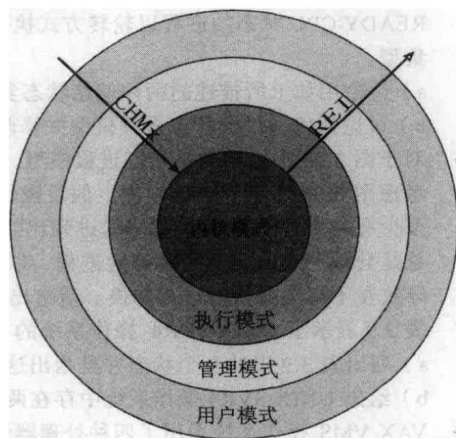


图 3.18 VAX/VMS 的访问模式